面向 Linux 内核的 LLM 辅助缺陷分析增强研究（基于 KernelGPT）开题报告

摘要

面向 Linux 内核的模糊测试技术（以 Syzkaller 为代表）已具备较强的缺陷发现能力，但在崩溃日志解释、根因定位与修复建议生成环节仍高度依赖人工，形成“发现快、分析慢”的工程瓶颈。本文以 KernelGPT 为基座，围绕其“规格生成—校验修复—融合 fuzz”流程，提出面向开题阶段的工程化改造与验证方案：通过低资源构建策略、镜像兼容性修复与批量规格筛选，打通可复现的 crash 数据生产链路；在此基础上，设计“日志精简+静态上下文检索+LLM 诊断”的自动化根因分析路线。当前阶段已完成三批规格生成与评估（78 条生成，62 条通过），并在 16G 内存开发机上稳定完成 30 分钟融合 fuzz 任务。该结果为后续开展崩溃语义分析准确性评测与自动化诊断报告生成提供了可用数据基础与可复现实验环境。

关键词

Linux 内核；Syzkaller；KernelGPT；大语言模型；根因分析；模糊测试

一、项目研究意义

Linux 内核模糊测试在“发现缺陷”上已经高度自动化，但在“分析缺陷”上仍存在明显断层。当前研究流程中，Syzkaller 能快速产出 crash、reproducer 与调用栈信息；然而真正耗时的步骤是对 panic/oops 日志进行人工解读、关联源码上下文并完成根因归纳。这一矛盾导致漏洞研究的产出效率受限，形成“发现快、分析慢”的工程瓶颈。

本课题的目标是将 KernelGPT 的能力从“生成可 fuzz 的高质量输入”进一步扩展到“自动化理解 crash”。具体而言，围绕崩溃日志构建“证据提取—上下文检索—LLM 诊断”链路，输出结构化、可验证的诊断结果，降低人工 triage 成本，并为后续修复验证提供高质量输入。

二、相关现状的调研

2.1 内核缺陷挖掘现状

以 Syzkaller 为代表的覆盖引导型模糊测试已成为 Linux 内核漏洞发现的事实标准。其优势是自动化程度高、复现能力强、社区工具成熟。但这一路线主要解决“是否能触发异常”，对“为何触发异常”支持有限。

2.2 KernelGPT 的能力边界与切入点

KernelGPT 通过静态分析抽取上下文，并利用 LLM 生成/修复 syscall 规格，显著提高了 fuzz 输入有效性。该项目已具备稳定的数据生产能力（spec、log、reproducer），非常适合作为后续自动化根因分析研究的基座。其当前边界在于：缺少针对 crash 的自动归因与可读化报告模块。

2.3 LLM 融合分析的机会与风险

LLM 在非结构化日志理解、跨证据归纳方面有优势，但直接喂入长日志容易出现幻觉、遗漏关键约束。为此，需要用静态分析产物提供语义锚点，避免“仅日志推理”导致的错误结论。

三、所选择工具初步搭建情况

3.1 基线链路打通

本课题严格参考项目 README 的标准流程完成基线搭建与验证，形成了可复现实验链路：

1. **环境与依赖准备**：完成仓库拉取、子模块初始化（`linux/` 与 `syzkaller/`）、Python 依赖安装，并配置 Clang 14、`bear`、`make` 等工具，确保静态分析与内核构建环境一致；
2. **内核准备与编译数据库生成（README Step 1）**：在 `linux/` 下执行 `allyesconfig`，并通过 `bear -- make ...` 生成 `compile_commands.json`，为后续静态分析提供编译上下文；对于 README 标注的特定提交编译问题，已结合补丁流程完成兼容处理；
3. **静态分析工具构建与产物生成**：在 `spec-gen/analyzer/` 编译 `analyze` 与 `usage`，并依次执行分析与处理脚本，稳定产出 `processed_handlers.json`、`processed_func.json` 等 `processed_*.json` 中间产物；
4. **规格生成**：在 `spec-gen/` 侧完成 LLM 配置后，使用 `gen_spec.py` 基于 `processed_handlers.json` 生成候选规格（JSON）；
5. **规格校验与修复**：使用 `eval_spec.py` 调用 `spec-eval/run-specs.py` 执行语法/常量/描述检查，自动修复并筛选可用规格，形成可直接复用的 `correct-spec` 集合；
6. **融合运行与日志闭环**：通过 `eval_spec.py --merge` 将 JSON 规格转换并注入 Syzkaller，完成 `extract -> descriptions -> build -> run` 全流程，日志统一沉淀在 `spec-eval/debug/merged.log` 与 `workdir-default-*/default-*.log`。

在上述流程基础上，当前已实现“批量生成—批量筛选—融合运行”的稳定闭环，能够持续为后续改进点 1（基于 LLM 的自动化崩溃根因分析）提供可追踪输入数据。

3.2 LLM API 接口可扩展性改造

3.2.1 改造缘由

结合前期排查记录和项目代码现状，原始流程存在两类问题：

1. **成本问题**：默认依赖 OpenAI 官方模型，批量 spec 生成时 token 消耗高；
2. **模型下架**：其调用的gpt-4-1106-preview目前已经下架，无法再使用，必须进行更换

为支撑后续大量 crash 样本分析，本课题需要将 LLM 层从“单模型绑定”改为“可配置后端 + 兼容多服务”，以兼顾成本、稳定性与可复现实验需求。

3.2.2 实现过程

当前改造已在 `spec-gen/llm_utils.py` 落地，核心机制如下：

- **统一配置入口**：通过 `_get_llm_config()` 从环境变量读取 provider/model/base_url/max_tokens 等参数；
- **兼容 OpenAI 与 OpenAI-compatible**：`OpenAI(**client_kwargs)` 方式支持 `base_url` 注入，可直接接入兼容协议的低成本模型服务；目前接入了DeepSeek的api用于规格生成和修复。
- **鲁棒性增强**：
    - 对上下文超限异常自动降 `max_tokens` 重试；
    - 提供最大失败次数与重试间隔控制；

    - 支持 `response_format={"type":"json_object"}`，降低解析失败概率；
- **代理与网络适配**：支持 `trust_env` 并规范化 socks 代理环境变量，提升不同网络环境可用性。

3.2.3 当前效果

该改造已用于本次三批规格生成流程，验证了“保留原有调用接口不变、通过环境变量切换后端模型”的可行性。后续在 crash 自动分析链路中可复用同一套接口策略，减少工程重复。

3.3 机器资源与稳定性优化

由于原项目对于计算机资源的利用超过了个人电脑的配置，导致个人电脑卡死。针对 OOM 与环境不兼容问题，已实施以下改造：

- 降低构建并发并限制 Go 包并发（`-j 1` + `GOFLAGS=-p=1`）；
- full build 仅保留运行必需目标（manager/fuzzer/execprog/executor）；
- `extract` 阶段统一设置 `CI=1`，规避非交互环境异常；
- 运行参数降载（`procs=1`, `vm.count=1`, `vm.cpu=1`）；
- 镜像由 `bullseye/bookworm` 迁移至 `trixie`，修复 `syz-executor version` 兼容问题。

3.4 关键运行日志摘录（plaintext）

以下摘录用于证明“构建通过 + fuzz 正常运行 + 正常收尾”：

```
2026-04-04 19:14:15.267 | INFO  | Done: extract constatns for .../spec-eval/debug/default-tmp/syzkaller
2026-04-04 19:14:20.012 | INFO  | Done: generate descriptions for .../spec-eval/debug/default-tmp/syzkaller
2026-04-04 19:15:07.571 | INFO  | Done: build syzkaller (tools/syz-env make executor TARGETOS=linux TARGETARCH=amd64 -j 1)
2026-04-04 19:15:07.575 | INFO  | Started syzkaller for default, PID: 4039149
2026-04-04 19:15:07.575 | INFO  | Fuzzing time is: 30 mins
2026-04-04 19:47:08.257 | INFO  | Fuzzing done, bye.
```

```
2026/04/04 19:18:26 VMs 1, executed 2219, cover 3958, signal 4501/5325, crashes 0, repro 0
2026/04/04 19:25:16 VMs 1, executed 12205, cover 6835, signal 8188/9066, crashes 0, repro 0
2026/04/04 19:30:26 VMs 1, executed 20894, cover 7266, signal 9363/10690, crashes 0, repro 0
```

3.5 批量规格评估结果

表 1 给出三批规格的评估统计结果。

表1 批量规格评估统计	生成数量	校验通过（return=0）	失败数量（非0）	通过率
b1	22	22	0	100.0%
b2	22	18	4	81.8%

表1 批量规格评估统计	生成数量	校验通过（return=0）	失败数量（非0）	通过率
b3	34	22	12	64.7%
合计	78	62	16	79.5%

说明：通过规格已合并用于后续融合 fuzz；触发合并失败的样本已归档至 `spec-gen/spec-merge-failed/`，用于后续失败模式分析与自动修复策略研究。

3.6 当前可复现实验配置

```
# spec-eval/template-valid.cfg
procs: 1
vm.count: 1
vm.cpu: 1
vm.mem: 2048
image: __IMAGE__/trixie.img
sshkey: __IMAGE__/trixie.id_rsa
```

```
# spec-eval/run-specs.py 关键策略
extract 阶段: tools/syz-env make extract ... (CI=1)
descriptions 阶段: make descriptions
full build 阶段: manager/fuzzer/execprog/executor (最小集合)
并发控制: make -j 1 + GOFLAGS=-p=1
```

3.7 运行截图

syzkaller

Stats:

revision	500bfdc4
config	config
uptime	8m17s
fuzzing	6m0s
corpus	236
triage queue	0
signal	7069
coverage	5841
syscalls	92
buffer too small	0 (0/hour)
crash types	0 (0/hour)
crashes	0 (0/hour)
exec candidate	0 (0/hour)
exec collide	2202 (332/min)
exec fuzz	0 (0/hour)
exec gen	1 (9/hour)
exec hints	197 (29/min)
exec minimize	4496 (11/sec)
exec seeds	11 (99/hour)
exec smash	1138 (171/min)
exec total	9110 (22/sec)
exec triage	1065 (160/min)
executor restarts	1 (9/hour)
max signal	7791 (19/sec)
new inputs	252 (38/min)
rotated inputs	0 (0/hour)
suppressed	0 (0/hour)
vm restarts	1 (9/hour)

四、进度安排与预期成效

4.1 阶段化进度安排

阶段 A：崩溃日志结构化提取

目标：从 Syzkaller 原始输出中抽取 RCA 所需最小证据集。

- 输入：`workdir-*/crashes/*/{description,report*,log*,repro*}`；
- 处理：提取崩溃类型、关键函数、调用栈片段、文件行号、触发 syscall 片段；
- 输出：统一 JSON 模板（用于后续检索与提示词生成）。

可行性依据：当前 fuzz 链路已稳定，日志与 crash 目录结构固定，可直接批处理。

阶段 B：静态上下文检索融合

目标：为崩溃证据补足源码语义锚点，降低 LLM 幻觉。

- 输入：阶段 A 的结构化 crash 条目 + `spec-gen/analyzer/processed_*.json`；
- 处理：按函数名/文件路径检索源码片段、相关结构体与关键字段定义；
- 输出："日志证据 + 静态上下文"联合样本。

可行性依据：KernelGPT 现有静态分析产物已可稳定生成，且已在 spec 生成阶段验证可用。

阶段 C：LLM 诊断链路实现

目标：形成可批量运行的自动化诊断原型。

- 输入：阶段 B 联合样本；
- 处理：
    - 构建结构化提示词模板（根因、证据、影响范围、验证建议）；
    - 复用已改造的可配置 LLM 接口（模型可替换、参数可控）；
    - 加入格式校验与失败重试。
- 输出：结构化诊断报告（JSON/Markdown）。

可行性依据：LLM 接口层已完成可扩展改造，支持低成本模型与批量调用。

阶段 D：评估与报告固化

目标：量化改进效果并形成开题后可持续迭代基线。

- 指标：根因定位一致性、报告可读性、人工分析时长下降比例；
- 方法：与人工分析/历史修复记录对照，统计正确率与可操作性；

4.2 预期成效

1. 一套挂载于 KernelGPT 的 crash 自动分析原型工具链；
2. 面向典型内核崩溃场景的结构化诊断报告生成能力；
3. 面向 16G 开发机可复现的运行脚本与参数模板；

五、参考文献

[1] Yang C, Zhao Z, Zhang L. KernelGPT: Enhanced Kernel Fuzzing via Large Language Models[C]//Proceedings of ASPLOS 2025. ACM, 2025: 560-573. DOI: 10.1145/3676641.3716022.

[2] Google. Syzkaller: Kernel Fuzzer[EB/OL]. https://github.com/google/syzkaller

[3] Linux Kernel Organization. The Linux Kernel Archives[EB/OL]. https://www.kernel.org/

[4] Clang Team. Clang 14 Documentation[EB/OL]. https://clang.llvm.org/docs/

[5] OpenAI. API Reference[EB/OL]. https://platform.openai.com/docs/api-reference

[6] DeepSeek. API Documentation[EB/OL]. https://api-docs.deepseek.com/